🎓 Event Management Platform

Complete Code Explanation — Presentation Guide

1. Project Overview

What is it? A full-stack **College Event Management Platform** built with the **MERN stack** (MongoDB, Express.js, React, Node.js).

What does it do? It lets college students browse and register for events, club admins create events and manage volunteers, and college admins approve/reject requests — all in one webapp.

Key highlights:

- Role-based access control (4 user roles)
- QR code-based attendance tracking
- Auto-generated PDF participation certificates
- Notification / inbox system
- Responsive UI with sidebar navigation

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	Fast UI with component-based architecture
Styling	Tailwind CSS 4 + shadcn/ui	Modern, responsive design system
Backend	Express.js (Node.js)	REST API server
Database	MongoDB (via Mongoose)	NoSQL document storage
Authentication	JWT (JSON Web Tokens)	Secure, stateless authentication
File Uploads	Multer	Event poster image uploads
QR Scanning	@zxing/library	Camera-based QR code reading
QR Generation	qrcode.react	Generates QR codes for event tickets
Certificates	jsPDF	Client-side PDF generation
Notifications	Sonner (toast library)	User feedback popups

3. Project Structure

```
Event Management Platform/
├── backend/                    ← Express.js API server
│   ├── config/db.js            ← MongoDB connection (with retry logic)
│   ├── middleware/auth.js      ← JWT authentication middleware
│   ├── models/                 ← 7 Mongoose database schemas
│   │   ├── User.js             ← User accounts & roles
│   │   ├── Event.js            ← Event details
│   │   ├── Club.js             ← Student clubs
│   │   ├── Registration.js     ← Event registrations
│   │   ├── Attendance.js       ← QR-scanned attendance logs
│   │   ├── Notification.js     ← Inbox messages
│   │   └── Request.js          ← Approval requests
│   ├── routes/                 ← 9 API endpoint files
│   │   ├── auth.js             ← Login / Register / Get current user
│   │   ├── events.js           ← CRUD for events
│   │   ├── clubs.js            ← Club listing & creation
│   │   ├── registrations.js    ← Event registration
│   │   ├── attendance.js       ← Mark & retrieve attendance
│   │   ├── notifications.js    ← Inbox & messaging
│   │   ├── requests.js         ← Approval workflow
│   │   ├── users.js            ← Profile & volunteer mgmt
│   │   └── upload.js           ← Image upload (multer)
│   ├── seed.js                 ← Database seeder (test data)
│   └── server.js               ← Main entry point
│
├── frontend/                   ← React + Vite application
│   └── src/
│       ├── main.jsx            ← React entry point
│       ├── app/
│       │   ├── App.jsx         ← Root component (auth guard)
│       │   ├── context/AuthContext.jsx  ← Global auth state
│       │   ├── utils/
│       │   │   ├── api.js              ← Centralized HTTP client
│       │   │   ├── storage.js          ← localStorage fallback
│       │   │   └── CertificateGenerator.js ← PDF certificate
│       │   └── components/     ← 15 page components + UI kit
│
├── package.json                ← Root: concurrent dev script
└── README.md                   ← Setup instructions
```

4. User Roles & Permissions

Role	Who	What they can do
student	Regular college student	Browse events, register, view inbox, create club requests, view profile
volunteer	Assigned by club admin	Everything a student can do + a "Volunteering" dashboard
clubAdmin	Runs a college club	Create events, scan QR for attendance, assign volunteers, message participants, manage "My Events"
collegeAdmin	College administrator	Approve/reject club creation & event requests

> ***Role transitions:*** *A student can request to create a club → if approved by college admin, they become a* `clubAdmin` *. A club admin can assign* `volunteer` *roles to students, and even transfer their admin position to another user.*

5. Backend — Explained in Detail

5.1 Server Entry Point — `server.js`

This is where the Express app starts. It:

1. Loads environment variables from `.env` (`MONGO_URI` , `JWT_SECRET` , `PORT`)
2. Connects to MongoDB via `connectDB()`
3. Sets up middleware: **CORS**, **JSON parsing**, and **static file serving** for uploaded images
4. Mounts all 9 route modules under `/api/...`
5. Starts listening on port **5001**

5.2 Database Connection — `config/db.js`

Connects to MongoDB using Mongoose with a **retry mechanism** — it tries up to 5 times with 5-second delays if the connection fails.

5.3 Auth Middleware — `middleware/auth.js`

This middleware is used on **protected routes**. It:

1. Extracts the JWT token from the `Authorization: Bearer <token>` header
2. Verifies the token using `jwt.verify()` with the secret key
3. Looks up the user in the database (excluding password)
4. Attaches the user object to `req.user` so route handlers can access it
5. Returns `401 Unauthorized` if no token or invalid token

5.4 Database Models (Schemas)

User.js

Field	Type	Description
`name`	String	Full name
`email`	String	Unique, lowercase login email
`password`	String	Hashed with bcrypt (min 6 chars)
`role`	Enum	`student`, `volunteer`, `clubAdmin`, or `collegeAdmin`
`semester`	String	Student's current semester
`department`	String	Academic department
`phoneNumber`	String	Contact number
`eventsParticipated`	Number	Count of attended events
`registeredEvents`	[ObjectId]	Array of event references

Special features:

- **Pre-save hook**: Automatically hashes password with bcrypt (salt factor 10) before saving
- **`matchPassword()` method**: Compares entered password against stored hash during login

Event.js

Field	Type	Description
`title` , `description`	String	Event info
`date` , `time`	String	When the event happens
`venue`	String	Location
`posterUrl`	String	Uploaded poster image URL
`clubId`	ObjectId → Club	Which club organized it
`clubName`	String	Denormalized club name
`maxParticipants`	Number	Capacity limit
`currentParticipants`	Number	Current registration count
`status`	Enum	`pending` → `approved` or `rejected`
`category`	String	e.g., "Technical", "Cultural", "Sports"
`createdBy`	ObjectId → User	Club admin who created it
`isPast`	Boolean	Auto-set when event date passes

Club.js

Stores club name, description, admin reference (`adminId` , `adminName`), approval status, contact email, and member count.

Registration.js

Links a `userId` to an `eventId` with embedded `userData` (name, semester, department, phone). Has `attended` (boolean) and `attendedAt` (timestamp) fields updated when QR is scanned. A **compound unique index** on `{userId, eventId}` prevents duplicate registrations.

Attendance.js

Records each QR scan: `userId` , `eventId` , `scannedBy` (who scanned), and `timestamp` .

Notification.js

Inbox messages with `type` (ticket / announcement / volunteer), `subject` , `content` , linked `eventId` , and `read` status.

Request.js

Approval workflow items with `type` (event / club), `title`, `description`, `submittedBy`, `status` (pending / approved / rejected), and `relatedId` (reference to the actual event or club).

5.5 API Routes (Endpoints)

auth.js — Authentication

Method	Endpoint	What it does
`POST`	`/api/auth/register`	Creates a new user account, returns JWT token
`POST`	`/api/auth/login`	Validates credentials, returns JWT token
`GET`	`/api/auth/me`	Returns currently logged-in user's data (requires token)

Token is generated with `jwt.sign()` and expires in **30 days**.

events.js — Event Management

Method	Endpoint	What it does
`GET`	`/api/events`	Returns all approved upcoming events (auto-marks past events)
`GET`	`/api/events/past`	Returns past events
`GET`	`/api/events/my-events`	Club admin's own events
`GET`	`/api/events/:id`	Single event details
`POST`	`/api/events`	Create a new event (status starts as `pending`)
`PUT`	`/api/events/:id`	Update event (owner or college admin only)

Smart feature: On every GET request, it automatically runs `updateMany()` to mark events with past dates as `isPast: true` .

registrations.js — Event Registration

Method	Endpoint	What it does
`POST`	`/api/registrations`	Register for an event (checks duplicates & capacity)
`GET`	`/api/registrations/user`	Get logged-in user's registrations with event details
`GET`	`/api/registrations/event/:eventId`	Get all registrations for an event
`GET`	`/api/registrations/check/:eventId`	Check if current user is registered

When a student registers, the system: creates the registration record, increments the event's participant count, adds to the user's registered events array, and creates a ticket notification.

attendance.js — QR Attendance

Method	Endpoint	What it does
`POST`	`/api/attendance`	Mark attendance (QR scan)
`GET`	`/api/attendance/event/:eventId`	Get attendance list for an event
`GET`	`/api/attendance/today`	Get today's attendance records

When attendance is marked: validates registration exists, checks it hasn't been scanned already, marks `attended=true` on the registration, creates an attendance record, and increments `eventsParticipated` on the user.

clubs.js — Club Management

Method	Endpoint	What it does
`GET`	`/api/clubs`	List all approved clubs
`POST`	`/api/clubs`	Create a club (starts as `pending`)

notifications.js — Messaging

Method	Endpoint	What it does
`GET`	`/api/notifications`	Get logged-in user's notifications
`PUT`	`/api/notifications/:id/read`	Mark a notification as read
`POST`	`/api/notifications/volunteer`	Club admin sends message to all volunteers
`POST`	`/api/notifications/event-message`	Club admin sends message to event participants

requests.js — Approval Workflow

Method	Endpoint	What it does
`GET`	`/api/requests`	College admin gets all pending requests
`POST`	`/api/requests`	Submit a new request (event or club)
`PUT`	`/api/requests/:id`	Approve or reject a request

When approved:

- **Club request** → Club status set to `approved`, submitter's role upgraded to `clubAdmin`
- **Event request** → Event status set to `approved` (becomes visible to students)

users.js — User Management

Method	Endpoint	What it does
`GET`	`/api/users/profile`	Get current user's profile
`PUT`	`/api/users/profile`	Update profile (name, semester, department, phone)
`GET`	`/api/users/students`	List all students and volunteers
`PUT`	`/api/users/transfer-admin`	Transfer club admin role to another user
`PUT`	`/api/users/:id/role`	Assign/remove volunteer role

upload.js — File Upload

Uses **Multer** to accept image uploads (JPEG, PNG, GIF, WebP, max 5MB), saves to the `uploads/` directory with unique timestamped filenames.

5.6 Seed Script — `seed.js`

Populates the database with test data:

- **6 users**: 3 club admins, 1 student, 1 volunteer, 1 college admin
- **3 clubs**: Tech Club, Cultural Club, Sports Club (all approved)
- **4 upcoming events**: Tech Symposium, Cultural Fest, Hackathon, Sports Day
- **2 past events**: Web Dev Workshop, Dance Competition

6. Frontend — Explained in Detail

6.1 Entry Point

- `main.jsx` — Renders the `<App />` component into the DOM
- `App.jsx` — Wraps everything in `<AuthProvider>`, shows a loading spinner during auth check, shows `<AuthPage>` if not logged in, or `<MainLayout>` if logged in

6.2 Auth Context — `AuthContext.jsx`

Global state manager for authentication:

- On app load: checks if a JWT token exists in `localStorage`, validates it via `GET /api/auth/me`
- Provides `login()`, `register()`, `logout()`, `updateProfile()` functions
- All components access auth state via `useAuth()` hook

6.3 API Client — `api.js`

Centralized HTTP client with a base `apiFetch()` wrapper that:

- Automatically attaches `Authorization: Bearer <token>` header
- Sets `Content-Type: application/json`
- Handles error responses
- Exports ~25 named functions for each API endpoint

6.4 Certificate Generator — `CertificateGenerator.js`

Generates a professional **landscape A4 PDF certificate** using jsPDF with decorative borders, gold accent lines, student name, event name, club name, date, venue. Auto-downloads as `Certificate_<EventName>.pdf`.

6.5 Page Components (15 total)

AuthPage — Login / Register

A tabbed form allowing users to log in or register. New users always register as `student` (role field is fixed/disabled) — other roles are assigned later through the approval or assignment

workflow.

MainLayout — Navigation Hub

The main shell featuring a **sidebar** with role-filtered navigation items, **mobile-responsive** hamburger menu, state-based page routing, and user info in the header.

Dashboard — Upcoming Events Grid

Displays all approved upcoming events in a responsive card grid with poster images, category badges, search, filter by category, and a progress bar showing registration fill rate.

EventDetails — Event View + Registration

Shows full event details. Students fill in registration form (semester, department, phone). After registering, generates a QR code (using `qrcode.react`) containing `userId|eventId` . Club admins see a participant list and can send messages.

RegisteredEvents — My Registrations

Lists all events the user has registered for, showing registration date and attended status.

PastEvents — Past Events + Certificates

Shows completed events with attendance watermarks (ATTENDED/NOT ATTENDED). If the user attended, they get a **"Download Certificate"** button that generates the PDF.

MyEvents — Club Admin's Events

Shows all events created by the club admin, regardless of approval status, with participant counts.

EventCreation — Create Event Form

Form with: title, description, date, time, venue, category (Technical/Cultural/Sports/Academic/Social), max participants, and poster upload. Automatically creates both the event AND an approval request.

ClubCreation — Create Club Form

Form for students to request a new club: name, description, contact email, member count. Creates both the club (as pending) and an approval request.

Inbox — Notification Center

Shows all notifications (tickets, announcements, volunteer messages) with read/unread status.

Profile — User Profile Editor

Displays and allows editing of: name, semester, department, phone number, and events participated count.

QRScanner — Attendance Scanner

Uses device camera via `@zxing/library` to scan QR codes. Parses `userId|eventId`, calls attendance API, shows success/error toasts. Also displays today's scanned records.

AssignVolunteers — Volunteer Management

Club admins view all students, assign/remove volunteer role, send messages to volunteers, and transfer admin position.

VolunteerDashboard — Volunteer View

Shows volunteer-specific information and assignment status.

RequestsPage — Admin Approval Panel

College admins see pending requests with approve/reject buttons. Approving a club upgrades the submitter to `clubAdmin`.

7. Key Application Flows

7.1 Login Flow

1. User enters email + password on AuthPage
2. Frontend sends `POST /api/auth/login`
3. Backend finds user by email in MongoDB
4. Backend runs `bcrypt.compare(password, hash)`
5. Backend generates JWT with `jwt.sign(userId)`
6. Frontend stores token in `localStorage` , sets user in AuthContext
7. App renders MainLayout → Dashboard

7.2 Event Registration Flow

1. Student clicks "Register" on EventDetails page
2. Frontend sends `POST /api/registrations { eventId, userData }`
3. Backend checks for duplicate registration
4. Backend checks event capacity (not full)
5. Backend creates Registration + increments participant count
6. Backend pushes eventId to user's registeredEvents array
7. Backend creates a ticket Notification
8. Frontend shows QR code (`userId|eventId`) for attendance

7.3 QR Attendance Flow

1. Club admin opens QR Scanner, activates camera
2. Camera scans student's QR code → parses `userId|eventId`
3. Frontend sends `POST /api/attendance { userId, eventId }`
4. Backend validates registration exists & not already scanned

5. Backend sets `registration.attended = true`
6. Backend creates Attendance record + increments `eventsParticipated`
7. Frontend shows success toast with student name

7.4 Club/Event Approval Flow

1. Student/Club Admin creates Club or Event (status: `pending`)
2. Also creates a Request (`POST /api/requests`) for admin review
3. College Admin views all Requests on RequestsPage
4. College Admin clicks Approve → `PUT /api/requests/:id { status: approved }`
5. Backend updates Club/Event status to `approved`
6. If club: submitter's role is upgraded to `clubAdmin`

8. How to Run the Project

```
# 1. Install dependencies (in root folder)
npm install

# 2. Install dependencies in both frontend and backend
cd backend && npm install
cd ../frontend && npm install
cd ..

# 3. Seed the database with test data
npm run seed

# 4. Start both servers concurrently
npm run dev
```

Test Accounts (all password: `password123`)

Role	Email
College Admin	admin@college.edu
Club Admin	john@college.edu
Student	student@college.edu
Volunteer	volunteer@college.edu

9. Key Presentation Talking Points

1. **MERN Stack Architecture** — Separate frontend (React/Vite on port 5173) and backend (Express on port 5001) communicating via REST APIs
2. **Security** — JWT-based authentication, bcrypt password hashing, role-based route protection
3. **QR Attendance System** — Students get a unique QR code on registration; club admins scan it with their camera to mark attendance

4. **Certificate Generation —** Professional PDF certificates generated entirely client-side using jsPDF
5. **Approval Workflow —** Two-step process: club admins create events → college admin approves them
6. **Role Escalation —** Students can create clubs → become club admins upon approval
7. **Notifications —** Ticket confirmation, volunteer messages, event announcements all delivered through the inbox
8. **File Upload —** Event posters uploaded via Multer, served as static files
9. **Database Seeding —** One `npm run seed` command populates realistic test data
10. **Responsive Design —** Works on both desktop (sidebar) and mobile (hamburger menu)